# Student Setup Guide

## 1. Review your web application(s)

Review your URLs that you are using for invoking your services. These need to have be setup for a production deployment in AWS vs the developer environment. AWS Load Balancers use a listener route so for production these routes need to be defined in your application.

| API path | Target group | Backend port |
|---|---|---|
| **/api/orders/*** | leap-orders-tg | 8090 |
| **/api/auth/*** | leap-auth-tg | 3000 |
| **/api/holdings/*** | leap-holdings-tg | 6200 |
| **/api/reporting/*** | Leap-reporting-tg | 7200 |

Traffic flow: Browser (HTTPS) → CloudFront (/api/* behavior) → ALB (HTTP:80 for this lab) → ECS

Example: Pre-Established Paths for team load balancer(s)

| Rule priority | Path | Target group (stack resource) |
|---|---|---|
| 10 | `/api/orders`, `/api/orders/*` | `OrdersTarget` |
| 20 | `/api/auth`, `/api/auth/*` | `AuthTarget` |
| 30 | `/api/holdings`, `/api/holdings/*` | `HoldingsTarget` |
| 40 | `/api/reporting`, `/api/reporting/*` | `ReportingTarget` |

1. MaUp under your ./src directory for your web application, create an environments folder. In this folder create two files, one named environment.ts and environment.production.ts. The environment.ts is used for the development environment and the other is used for product.

2. Update the environment.product.ts:

```
export const environment = {
  ordersUrl: '/api/orders',
  authUrl: '/api/auth',
  holdingsUrl: '/api/holdings',
  reportingUrl: '/api/reporing'
};
```

3. Update the development.ts based on your current localhost service routing:

(Example: hello world reference application)

```
export const environment = {
  ordersUrl: 'http://localhost:8090',
  authUrl: 'http://localhost:3000',
  holdingsUrl: 'http://localhost:6200',
};
```

4. Update the references to the URLs in your associated service calls:

(Example: hello world reference application)

```
import { environment } from '../../environments/environment'

….

const ORDERS_SERVICE_BASE = environment.ordersUrl;
```

5. Update your angular.json to leverage the environment.production.ts.

In your production configuration add the the following:

```
            "fileReplacements": [
              {
               "replace": "src/environments/environment.ts",
               "with": "src/environments/environment.production.ts"
              }
            ],
```

6. Run a production build of your application: npx ng build --configuration production

## 2. Review your micro-service application(s)

Review your Dockerfiles for your micro-service applications. If you have java-based services ensure that your are skipping tests when running your build with maven:

```
RUN mvn -B clean package -DskipTests
```

For your python based service ensure that you are not running your unit tests via docker file:

```
# Run unit tests
# RUN python –m pytest –v
```

For your AWS deployments you will only be running your health checks. You will not be running SonarQube, unit tests, or e2d integration tests.